



Chapter 8

C Characters and Strings

C How to Program, 7/e



OBJECTIVES

In this chapter, you'll:

- Use the functions of the character-handling library (`<ctype.h>`).
- Use the string-conversion functions of the general utilities library (`<stdlib.h>`).
- Use the string and character input/output functions of the standard input/output library (`<stdio.h>`).
- Use the string-processing functions of the string-handling library (`<string.h>`).
- Use the memory-processing functions of the string-handling library (`<string.h>`).



8.1 Introduction

8.2 Fundamentals of Strings and Characters

8.3 Character-Handling Library

8.3.1 Functions `isdigit`, `isalpha`, `isalnum` and `isxdigit`

8.3.2 Functions `islower`, `isupper`, `tolower` and `toupper`

8.3.3 Functions `isspace`, `isctrl`, `ispunct`, `isprint` and `isgraph`

8.4 String-Conversion Functions

8.4.1 Function `strtod`

8.4.2 Function `strtol`

8.4.3 Function `strtoul`

8.5 Standard Input/Output Library Functions

8.5.1 Functions `fgets` and `putchar`

8.5.2 Function `getchar`

8.5.3 Function `sprintf`

8.5.4 Function `sscanf`



8.6 String-Manipulation Functions of the String-Handling Library

8.6.1 Functions `strcpy` and `strncpy`

8.6.2 Functions `strcat` and `strncat`

8.7 Comparison Functions of the String-Handling Library

8.8 Search Functions of the String-Handling Library

8.8.1 Function `strchr`

8.8.2 Function `strcspn`

8.8.3 Function `strpbrk`

8.8.4 Function `strrchr`

8.8.5 Function `strspn`

8.8.6 Function `strstr`

8.8.7 Function `strtok`

8.9 Memory Functions of the String-Handling Library

8.9.1 Function `memcpy`

8.9.2 Function `memmove`

8.9.3 Function `memcmp`

8.9.4 Function `memchr`

8.9.5 Function `memset`



8.10 Other Functions of the String-Handling Library

8.10.1 Function `strerror`

8.10.2 Function `strlen`

8.11 Secure C Programming



8.1 Introduction

- ▶ This chapter introduces the C Standard Library functions that facilitate string and character processing.
- ▶ The functions enable programs to process characters, strings, lines of text and blocks of memory.
- ▶ The chapter discusses the techniques used to develop editors, word processors, page-layout software, computerized typesetting systems and other kinds of text-processing software.
- ▶ The text manipulations performed by formatted input/output functions like `printf` and `scanf` can be implemented using the functions discussed in this chapter.



8.2 Fundamentals of Strings and Characters

- ▶ Characters are the fundamental building blocks of source programs.
- ▶ Every program is composed of a sequence of characters that—when grouped together meaningfully—is interpreted by the computer as a series of instructions used to accomplish a task.
- ▶ A program may contain **character constants**.
- ▶ A character constant is an `int` value represented as a character in single quotes.
- ▶ The value of a character constant is the integer value of the character in the machine's **character set**.



8.2 Fundamentals of Strings and Characters (Cont.)

- ▶ For example, 'z' represents the integer value of z, and '\n' the integer value of newline (122 and 10 in ASCII, respectively).
- ▶ A **string** is a series of characters treated as a single unit.
- ▶ A string may include letters, digits and various **special characters** such as +, -, *, / and \$.
- ▶ **String literals**, or **string constants**, in C are written in double quotation marks.



8.2 Fundamentals of Strings and Characters (Cont.)

- ▶ A string in C is an array of characters ending in the **null character** (`'\0'`).
- ▶ A string is accessed via a *pointer* to the first character in the string.
- ▶ The value of a string is the address of its first character.
- ▶ Thus, in C, it's appropriate to say that a **string is a pointer**—in fact, a pointer to the string's first character.
- ▶ In this sense, strings are like arrays, because an array is also a pointer to its first element.
- ▶ A *character array* or a *variable of type* `char *` can be initialized with a string in a definition.



8.2 Fundamentals of Strings and Characters (Cont.)

- ▶ The definitions

```
char color[] = "blue";  
const char *colorPtr = "blue";
```

each initialize a variable to the string "blue".

- ▶ The first definition creates a 5-element array `color` containing the characters 'b', 'l', 'u', 'e' and '\0'.
- ▶ The second definition creates pointer variable `colorPtr` that points to the string "blue" somewhere in memory.



Portability Tip 8.1

When a variable of type `char *` is initialized with a string literal, some compilers may place the string in a location in memory where it cannot be modified. If you might need to modify a string literal, it should be stored in a character array to ensure modifiability on all systems.



8.2 Fundamentals of Strings and Characters (Cont.)

- ▶ The preceding array definition could also have been written

```
char color[] = { 'b', 'l', 'u', 'e', '\0' };
```
- ▶ When defining a character array to contain a string, the array must be large enough to store the string *and* its terminating null character.
- ▶ The preceding definition automatically determines the size of the array based on the number of initializers in the initializer list.



Common Programming Error 8.1

Not allocating sufficient space in a character array to store the null character that terminates a string is an error.



Common Programming Error 8.2

Printing a “string” that does not contain a terminating null character is an error.



Error-Prevention Tip 8.1

When storing a string of characters in a character array, be sure that the array is large enough to hold the largest string that will be stored. C allows strings of any length to be stored. If a string is longer than the character array in which it's to be stored, characters beyond the end of the array will overwrite data in memory following the array.



8.2 Fundamentals of Strings and Characters (Cont.)

- ▶ A string can be stored in an array using `scanf`.
- ▶ For example, the following statement stores a string in character array `word[20]`:

```
scanf( "%s", word );
```
- ▶ The string entered by the user is stored in `word`.
- ▶ Variable `word` is an array, which is, of course, a pointer, so the `&` is not needed with argument `word`.



8.2 Fundamentals of Strings and Characters (Cont.)

- ▶ `scanf` will read characters until a space, tab, newline or end-of-file indicator is encountered.
- ▶ So, it's possible that, without the field width **19** in the conversion specifier **%19s**, the user input could exceed 19 characters and that your program might crash!
- ▶ For this reason, you should *always* use a field width when using `scanf` to read into a `char` array.
- ▶ The field width **19** in the preceding statement ensures that `scanf` reads a *maximum* of 19 characters and saves the last character for the string's terminating null character.



8.2 Fundamentals of Strings and Characters (Cont.)

- ▶ This prevents `scanf` from writing characters into memory beyond the end of `s`.
- ▶ (For reading input lines of arbitrary length, there's a nonstandard—yet widely supported—function `getline`, usually included in `stdio.h`.)
- ▶ For a character array to be printed properly as a string, the array must contain a terminating null character.



Common Programming Error 8.3

Processing a single character as a string. A string is a pointer—probably a respectably large integer. However, a character is a small integer (ASCII values range 0–255). On many systems this causes an error, because low memory addresses are reserved for special purposes such as operating-system interrupt handlers—so “access violations” occur.



Common Programming Error 8.4

Passing a character as an argument to a function when a string is expected (and vice versa) is a compilation error.



8.3 Character-Handling Library

- ▶ The **character-handling library** (`<ctype.h>`) includes several functions that perform useful tests and manipulations of character data.
- ▶ Each function receives an **unsigned char** (represented as an **int**) or **EOF** as an argument.
- ▶ **EOF** normally has the value -1 .
- ▶ Figure 8.1 summarizes the functions of the character-handling library.



Prototype	Function description
<code>int isblank(int c);</code>	Returns a true value if <i>c</i> is a <i>blank character</i> that separates words in a line of text and 0 (false) otherwise. [Note: This function is not available in Microsoft Visual C++.]
<code>int isdigit(int c);</code>	Returns a true value if <i>c</i> is a <i>digit</i> and 0 (false) otherwise.
<code>int isalpha(int c);</code>	Returns a true value if <i>c</i> is a <i>letter</i> and 0 otherwise.
<code>int isalnum(int c);</code>	Returns a true value if <i>c</i> is a <i>digit</i> or a <i>letter</i> and 0 otherwise.
<code>int isxdigit(int c);</code>	Returns a true value if <i>c</i> is a <i>hexadecimal digit character</i> and 0 otherwise. (See Appendix C for a detailed explanation of binary numbers, octal numbers, decimal numbers and hexadecimal numbers.)
<code>int islower(int c);</code>	Returns a true value if <i>c</i> is a <i>lowercase letter</i> and 0 otherwise.
<code>int isupper(int c);</code>	Returns a true value if <i>c</i> is an <i>uppercase letter</i> and 0 otherwise.
<code>int tolower(int c);</code>	If <i>c</i> is an <i>uppercase letter</i> , <code>tolower</code> returns <i>c</i> as a <i>lowercase letter</i> . Otherwise, <code>tolower</code> returns the argument unchanged.

Fig. 8.1 | Character-handling library (<ctype.h>) functions. (Part I of 2.)



Prototype	Function description
<code>int toupper(int c);</code>	If <i>c</i> is a <i>lowercase letter</i> , <code>toupper</code> returns <i>c</i> as an <i>uppercase letter</i> . Otherwise, <code>toupper</code> returns the argument unchanged.
<code>int isspace(int c);</code>	Returns a true value if <i>c</i> is a <i>whitespace character</i> —newline (<code>'\n'</code>), space (<code>' '</code>), form feed (<code>'\f'</code>), carriage return (<code>'\r'</code>), horizontal tab (<code>'\t'</code>) or vertical tab (<code>'\v'</code>)—and 0 otherwise.
<code>int iscntrl(int c);</code>	Returns a true value if <i>c</i> is a <i>control character</i> and 0 otherwise.
<code>int ispunct(int c);</code>	Returns a true value if <i>c</i> is a <i>printing character other than a space, a digit, or a letter</i> and returns 0 otherwise.
<code>int isprint(int c);</code>	Returns a true value if <i>c</i> is a <i>printing character including a space</i> and returns 0 otherwise.
<code>int isgraph(int c);</code>	Returns a true value if <i>c</i> is a <i>printing character other than a space</i> and returns 0 otherwise.

Fig. 8.1 | Character-handling library (`<ctype.h>`) functions. (Part 2 of 2.)



8.3.1 Functions `isdigit`, `isalpha`, `isalnum` and `isxdigit`

- ▶ Figure 8.2 demonstrates functions `isdigit`, `isalpha`, `isalnum` and `isxdigit`.
- ▶ Function `isdigit` determines whether its argument is a digit (0–9).
- ▶ Function `isalpha` determines whether its argument is an uppercase (A–Z) or lowercase letter (a–z).
- ▶ Function `isalnum` determines whether its argument is an uppercase letter, a lowercase letter or a digit.
- ▶ Function `isxdigit` determines whether its argument is a **hexadecimal digit** (A–F, a–f, 0–9).



```
1 // Fig. 8.2: fig08_02.c
2 // Using functions isdigit, isalpha, isalnum, and isxdigit
3 #include <stdio.h>
4 #include <ctype.h>
5
6 int main( void )
7 {
8     printf( "%s\n%s%s\n%s%s\n\n", "According to isdigit: ",
9         isdigit( '8' ) ? "8 is a " : "8 is not a ", "digit",
10         isdigit( '#' ) ? "# is a " : "# is not a ", "digit" );
11
12     printf( "%s\n%s%s\n%s%s\n%s%s\n\n",
13         "According to isalpha:",
14         isalpha( 'A' ) ? "A is a " : "A is not a ", "letter",
15         isalpha( 'b' ) ? "b is a " : "b is not a ", "letter",
16         isalpha( '&' ) ? "& is a " : "& is not a ", "letter",
17         isalpha( '4' ) ? "4 is a " : "4 is not a ", "letter" );
18 }
```

Fig. 8.2 | Using functions `isdigit`, `isalpha`, `isalnum` and `isxdigit`. (Part I of 3.)



```
19 printf( "%s\n%s%s\n%s%s\n%s%s\n\n",
20         "According to isalnum:",
21         isalnum( 'A' ) ? "A is a " : "A is not a ",
22         "digit or a letter",
23         isalnum( '8' ) ? "8 is a " : "8 is not a ",
24         "digit or a letter",
25         isalnum( '#' ) ? "# is a " : "# is not a ",
26         "digit or a letter" );
27
28 printf( "%s\n%s%s\n%s%s\n%s%s\n%s%s\n",
29         "According to isxdigit:",
30         isxdigit( 'F' ) ? "F is a " : "F is not a ",
31         "hexadecimal digit",
32         isxdigit( 'J' ) ? "J is a " : "J is not a ",
33         "hexadecimal digit",
34         isxdigit( '7' ) ? "7 is a " : "7 is not a ",
35         "hexadecimal digit",
36         isxdigit( '$' ) ? "$ is a " : "$ is not a ",
37         "hexadecimal digit",
38         isxdigit( 'f' ) ? "f is a " : "f is not a ",
39         "hexadecimal digit" );
40 } // end main
```

Fig. 8.2 | Using functions isdigit, isalpha, isalnum and isxdigit. (Part 2 of 3.)



According to `isdigit`:

8 is a digit

is not a digit

According to `isalpha`:

A is a letter

b is a letter

& is not a letter

4 is not a letter

According to `isalnum`:

A is a digit or a letter

8 is a digit or a letter

is not a digit or a letter

According to `isxdigit`:

F is a hexadecimal digit

J is not a hexadecimal digit

7 is a hexadecimal digit

\$ is not a hexadecimal digit

f is a hexadecimal digit

Fig. 8.2 | Using functions `isdigit`, `isalpha`, `isalnum` and `isxdigit`. (Part 3 of 3.)



8.3.1 Functions `isdigit`, `isalpha`, `isalnum` and `isxdigit` (Cont.)

- ▶ Figure 8.2 uses the conditional operator (`?:`) to determine whether the string `" is a "` or the string `" is not a "` should be printed in the output for each character tested.
- ▶ For example, the expression
`isdigit( '8' ) ? "8 is a " : "8 is not a "`
indicates that if `'8'` is a digit, the string `"8 is a "` is printed, and if `'8'` is not a digit (i.e., `isdigit` returns `0`), the string `"8 is not a "` is printed.



8.3.1 Functions `islower`, `isupper`, `tolower` and `toupper`

- ▶ Figure 8.3 demonstrates functions `islower`, `isupper`, `tolower` and `toupper`.
- ▶ Function `islower` determines whether its argument is a lowercase letter (a–z).
- ▶ Function `isupper` determines whether its argument is an uppercase letter (A–Z).
- ▶ Function `tolower` converts an uppercase letter to a lowercase letter and returns the lowercase letter.



8.3 Character-Handling Library (Cont.)

- ▶ If the argument is *not* an uppercase letter, `tolower` returns the argument *unchanged*.
- ▶ Function `toupper` converts a lowercase letter to an uppercase letter and returns the uppercase letter.
- ▶ If the argument is *not* a lowercase letter, `toupper` returns the argument *unchanged*.



```
1 // Fig. 8.3: fig08_03.c
2 // Using functions islower, isupper, tolower, toupper
3 #include <stdio.h>
4 #include <ctype.h>
5
6 int main( void )
7 {
8     printf( "%s\n%s%s\n%s%s\n%s%s\n%s\n\n",
9         "According to islower:",
10         islower( 'p' ) ? "p is a " : "p is not a ",
11         "lowercase letter",
12         islower( 'P' ) ? "P is a " : "P is not a ",
13         "lowercase letter",
14         islower( '5' ) ? "5 is a " : "5 is not a ",
15         "lowercase letter",
16         islower( '!' ) ? "! is a " : "! is not a ",
17         "lowercase letter" );
18 }
```

Fig. 8.3 | Using functions islower, isupper, tolower and toupper. (Part I of 3.)



```
19 printf( "%s\n%s%s\n%s%s\n%s%s\n%s%s\n\n",
20         "According to isupper:",
21         isupper( 'D' ) ? "D is an " : "D is not an ",
22         "uppercase letter",
23         isupper( 'd' ) ? "d is an " : "d is not an ",
24         "uppercase letter",
25         isupper( '8' ) ? "8 is an " : "8 is not an ",
26         "uppercase letter",
27         isupper( '$' ) ? "$ is an " : "$ is not an ",
28         "uppercase letter" );
29
30 printf( "%s%c\n%s%c\n%s%c\n%s%c\n",
31         "u converted to uppercase is ", toupper( 'u' ),
32         "7 converted to uppercase is ", toupper( '7' ),
33         "$ converted to uppercase is ", toupper( '$' ),
34         "L converted to lowercase is ", tolower( 'L' ) );
35 } // end main
```

Fig. 8.3 | Using functions islower, isupper, tolower and toupper. (Part 2 of 3.)



According to islower:

p is a lowercase letter

P is not a lowercase letter

5 is not a lowercase letter

! is not a lowercase letter

According to isupper:

D is an uppercase letter

d is not an uppercase letter

8 is not an uppercase letter

\$ is not an uppercase letter

u converted to uppercase is U

7 converted to uppercase is 7

\$ converted to uppercase is \$

L converted to lowercase is l

Fig. 8.3 | Using functions islower, isupper, tolower and toupper. (Part 3 of 3.)



8.3.3 Functions `isspace`, `isctrl`, `ispunct`, `isprint` and `isgraph`

- ▶ Figure 8.4 demonstrates functions `isspace`, `isctrl`, `ispunct`, `isprint` and `isgraph`.
- ▶ Function `isspace` determines whether a character is one of the following whitespace characters: space (' '), form feed ('\f'), newline ('\n'), carriage return ('\r'), horizontal tab ('\t') or vertical tab ('\v').



8.3.3 Functions `isspace`, `isctrl`, `ispunct`, `isprint` and `isgraph`

- ▶ Function `isctrl` determines whether a character is one of the following **control characters**: horizontal tab (`'\t'`), vertical tab (`'\v'`), form feed (`'\f'`), alert (`'\a'`), backspace (`'\b'`), carriage return (`'\r'`) or newline (`'\n'`).
- ▶ Function `ispunct` determines if a character is a **printing character** other than a space, a digit or a letter, such as `$`, `#`, `(`, `)`, `[`, `]`, `{`, `}`, `;`, `:` or `%`.
- ▶ Function `isprint` determines whether a character can be displayed on the screen (including the space character).
- ▶ Function `isgraph` is the same as `isprint`, except that the space character is not included.



```
1 // Fig. 8.4: fig08_04.c
2 // Using functions isspace, iscntrl, ispunct, isprint, isgraph
3 #include <stdio.h>
4 #include <ctype.h>
5
6 int main( void )
7 {
8     printf( "%s\n%s%s%s\n%s%s%s\n%s%s\n\n",
9         "According to isspace:",
10         "Newline", isspace( '\n' ) ? " is a " : " is not a ",
11         "whitespace character", "Horizontal tab",
12         isspace( '\t' ) ? " is a " : " is not a ",
13         "whitespace character",
14         isspace( '%' ) ? "% is a " : "% is not a ",
15         "whitespace character" );
16
17     printf( "%s\n%s%s%s\n%s%s\n\n", "According to iscntrl:",
18         "Newline", iscntrl( '\n' ) ? " is a " : " is not a ",
19         "control character", iscntrl( '$' ) ? "$ is a " :
20         "$ is not a ", "control character" );
21
```

Fig. 8.4 | Using functions `isspace`, `iscntrl`, `ispunct`, `isprint` and `isgraph`. (Part I of 3.)



```
22 printf( "%s\n%s%s\n%s%s\n%s%s\n\n",
23         "According to ispunct:",
24         ispunct( ';' ) ? "; is a " : "; is not a ",
25         "punctuation character",
26         ispunct( 'Y' ) ? "Y is a " : "Y is not a ",
27         "punctuation character",
28         ispunct( '#' ) ? "# is a " : "# is not a ",
29         "punctuation character" );
30
31 printf( "%s\n%s%s\n%s%s\n\n", "According to isprint:",
32         isprint( '$' ) ? "$ is a " : "$ is not a ",
33         "printing character",
34         "Alert", isprint( '\a' ) ? " is a " : " is not a ",
35         "printing character" );
36
37 printf( "%s\n%s%s\n%s%s\n\n", "According to isgraph:",
38         isgraph( 'Q' ) ? "Q is a " : "Q is not a ",
39         "printing character other than a space",
40         "Space", isgraph( ' ' ) ? " is a " : " is not a ",
41         "printing character other than a space" );
42 } // end main
```

Fig. 8.4 | Using functions isspace, iscntrl, ispunct, isprint and isgraph. (Part 2 of 3.)



According to isspace:

Newline is a whitespace character

Horizontal tab is a whitespace character

% is not a whitespace character

According to iscntrl:

Newline is a control character

\$ is not a control character

According to ispunct:

; is a punctuation character

Y is not a punctuation character

is a punctuation character

According to isprint:

\$ is a printing character

Alert is not a printing character

According to isgraph:

Q is a printing character other than a space

Space is not a printing character other than a space

Fig. 8.4 | Using functions isspace, iscntrl, ispunct, isprint and isgraph. (Part 3 of 3.)



8.4 String-Conversion Functions

- ▶ This section presents the [string-conversion functions](#) from the [general utilities library](#) (`<stdlib.h>`).
- ▶ These functions convert strings of digits to integer and floating-point values.
- ▶ Figure 8.5 summarizes the string-conversion functions.
- ▶ The C standard also includes `strtoll` and `strtoull` for converting strings to `long long int` and `unsigned long long int`, respectively.
- ▶ Note the use of `const` to declare variable `nPtr` in the function headers (read from right to left as “`nPtr` is a pointer to a character constant”); `const` specifies that the argument value will not be modified.



Function prototype	Function description
<code>double strtod(const char *nPtr, char **endPtr);</code>	Converts the string nPtr to double.
<code>long strtol(const char *nPtr, char **endPtr, int base);</code>	Converts the string nPtr to long.
<code>unsigned long strtoul(const char *nPtr, char **endPtr, int base);</code>	Converts the string nPtr to unsigned long.

Fig. 8.5 | String-conversion functions of the general utilities library.



8.4.1 Function `strtod`

- ▶ Function `strtod` (Fig. 8.6) converts a sequence of characters representing a floating-point value to double.
- ▶ The function returns 0 if it's unable to convert any portion of its first argument to double.
- ▶ The function receives two arguments—a string (`char *`) and a pointer to a string (`char **`).
- ▶ The string argument contains the character sequence to be converted to double—any whitespace characters at the beginning of the string are ignored.



8.4.1 Function strtod (Cont.)

- ▶ The function uses the `char **` argument to modify a `char *` in the calling function (`stringPtr`) so that it points to the *location of the first character after the converted portion of the string* or to the entire string if no portion can be converted.

- ▶ Line 14

```
d = strtod( string, &stringPtr );
```

indicates that `d` is assigned the double value converted from `string`, and `stringPtr` is assigned the location of the first character after the converted value (51.2) in `string`.



```
1 // Fig. 8.6: fig08_06.c
2 // Using function strtod
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main( void )
7 {
8     // initialize string pointer
9     const char *string = "51.2% are admitted"; // initialize string
10
11     double d; // variable to hold converted sequence
12     char *stringPtr; // create char pointer
13
14     d = strtod( string, &stringPtr );
15
16     printf( "The string \"%s\" is converted to the\n", string );
17     printf( "double value %.2f and the string \"%s\"\n", d, stringPtr );
18 }
```

The string "51.2% are admitted" is converted to the double value 51.20 and the string "% are admitted"

Fig. 8.6 | Using function strtod.



8.4.2 Function `strtol`

- ▶ Function `strtol` (Fig. 8.7) converts to `long int` a sequence of characters representing an integer.
- ▶ The function returns 0 if it's unable to convert any portion of its first argument to `long int`.
- ▶ The function receives three arguments—a string (`char *`), a pointer to a string and an integer.
- ▶ The string argument contains the character sequence to be converted to `double`—any whitespace characters at the beginning of the string are ignored.



8.4.2 Function `strtol` (Cont.)

- ▶ The function uses the `char **` argument to modify a `char *` in the calling function (`remainderPtr`) so that it points to the location of the first character after the converted portion of the string or to the entire string if no portion can be converted.
- ▶ The integer specifies the base of the value being converted. Line 13

```
x = strtol( string, &remainderPtr, 0 );
```

indicates that `x` is assigned the long value converted from `string`.



8.4.2 Function `strtol` (Cont.)

- ▶ The second argument, `remainderPtr`, is assigned the remainder of string after the conversion.
- ▶ Using `NULL` for the second argument causes the *remainder of the string to be ignored*.
- ▶ The third argument, `0`, indicates that the value to be converted can be in octal (base 8), decimal (base 10) or hexadecimal (base 16) format. The base can be specified as `0` or any value between 2 and 36.
- ▶ Numeric representations of integers from base 11 to base 36 use the characters `A–Z` to represent the values 10 to 35.



8.4.2 Function `strtol` (Cont.)

- ▶ For example, hexadecimal values can consist of the digits 0–9 and the characters A–F.
- ▶ A base-11 integer can consist of the digits 0–9 and the character A.
- ▶ A base-24 integer can consist of the digits 0–9 and the characters A–N.
- ▶ A base-36 integer can consist of the digits 0–9 and the characters A–Z.
- ▶ The function returns 0 if it's unable to convert any portion of its first argument to a `long int` value.



```
1 // Fig. 8.7: fig08_07.c
2 // Using function strtol
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main( void )
7 {
8     const char *string = "-1234567abc"; // initialize string pointer
9
10    char *remainderPtr; // create char pointer
11    long x; // variable to hold converted sequence
12
13    x = strtol( string, &remainderPtr, 0 );
14
15    printf( "%s\\\"%s\\\"\\n%s%ld\\n%s\\\"%s\\\"\\n%s%ld\\n",
16        "The original string is ", string,
17        "The converted value is ", x,
18        "The remainder of the original string is ",
19        remainderPtr,
20        "The converted value plus 567 is ", x + 567 );
21 }
```

Fig. 8.7 | Using function `strtol`. (Part 1 of 2.)



```
The original string is "-1234567abc"  
The converted value is -1234567  
The remainder of the original string is "abc"  
The converted value plus 567 is -1234000
```

Fig. 8.7 | Using function `strtol`. (Part 2 of 2.)



8.4.3 Function `strtoul`

- ▶ Function `strtoul` (Fig. 8.8) converts to unsigned long int a sequence of characters representing an unsigned long int value.
- ▶ The function works identically to function `strtol`.
The statement

```
x = strtoul( string, &remainderPtr, 0 );
```

in line 12 of Fig. 8.8 indicates that x is assigned the unsigned long int value converted from string.



8.4.3 Function `strtoul` (Cont.)

- ▶ The second argument, `&remainderPtr`, is assigned the remainder of `string` after the conversion.
- ▶ The third argument, `0`, indicates that the value to be converted can be in octal, decimal or hexadecimal format.



```
1 // Fig. 8.8: fig08_08.c
2 // Using function strtoul
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main( void )
7 {
8     const char *string = "1234567abc"; // initialize string pointer
9     unsigned long int x; // variable to hold converted sequence
10    char *remainderPtr; // create char pointer
11
12    x = strtoul( string, &remainderPtr, 0 );
13
14    printf( "%s\\\"%s\\\"\\n%s%lu\\n%s\\\"%s\\\"\\n%s%lu\\n",
15        "The original string is ", string,
16        "The converted value is ", x,
17        "The remainder of the original string is ",
18        remainderPtr,
19        "The converted value minus 567 is ", x - 567 );
20 } // end main
```

Fig. 8.8 | Using function strtoul. (Part I of 2.)



The original string is "1234567abc"
The converted value is 1234567
The remainder of the original string is "abc"
The converted value minus 567 is 1234000

Fig. 8.8 | Using function strtoul. (Part 2 of 2.)



8.5 Standard Input/Output Library Functions

- ▶ This section presents several functions from the standard input/output library (`<stdio.h>`) specifically for manipulating character and string data.
- ▶ Figure 8.12 summarizes the character and string input/output functions of the standard input/output library.



Function prototype	Function description
<code>int getchar(void);</code>	Inputs the next character from the standard input and returns it as an integer.
<code>char *fgets(char *s, int n, FILE *stream);</code>	Inputs characters from the specified stream into the array <i>s</i> until a <i>newline</i> or <i>end-of-file</i> character is encountered, or until <i>n</i> - 1 bytes are read. In this chapter, we specify the stream as <code>stdin</code> —the <i>standard input stream</i> , which is typically used to read characters from the keyboard. A <i>terminating null character</i> is appended to the array. Returns the string that was read into <i>s</i> .
<code>int putchar(int c);</code>	Prints the character stored in <i>c</i> and returns it as an integer.
<code>int puts(const char *s);</code>	Prints the string <i>s</i> followed by a <i>newline</i> character. Returns a nonzero integer if successful, or EOF if an error occurs.

Fig. 8.9 | Standard input/output library character and string functions. (Part I of 2.)



Function prototype	Function description
<code>int sprintf(char *s, const char *format, ...);</code>	Equivalent to <code>printf</code>, except the output is stored in the array <code>s</code> instead of printed on the screen. Returns the number of characters written to <code>s</code>, or EOF if an error occurs. [<i>Note:</i> We mention the more secure related functions <code>snprintf</code> and <code>snprintf_s</code> in the Secure C Programming section of this chapter and in Appendix F.]
<code>int sscanf(char *s, const char *format, ...);</code>	Equivalent to <code>scanf</code>, except the input is read from the array <code>s</code> rather than from the keyboard. Returns the number of items successfully read by the function, or EOF if an error occurs.

Fig. 8.9 | Standard input/output library character and string functions. (Part 2 of 2.)



8.5.1 Functions `fgets` and `putchar`

- ▶ Figure 8.10 uses functions `fgets` and `putchar` to read a line of text from the standard input (keyboard) and recursively output the characters of the line in reverse order.
- ▶ Function `fgets` reads characters from the *standard input* into its first argument—an array of `chars`—until a *newline* or the *end-of-file* indicator is encountered, or until the maximum number of characters is read.
- ▶ The maximum number of characters is one fewer than the value specified in `fgets`'s second argument.



8.5.1 Functions `fgets` and `putchar` (Cont.)

- ▶ The third argument specifies the *stream* from which to read characters—in this case, we use the *standard input stream* (`stdin`).
- ▶ A null character (`'\0'`) is appended to the array when reading terminates.
- ▶ Function `putchar` prints its character argument.
- ▶ The program calls *recursive* function `reverse` to print the line of text backward.
- ▶ If the first character of the array received by `reverse` is the null character `'\0'`, `reverse` returns.



8.5.1 Functions `fgets` and `putchar` (Cont.)

- ▶ Otherwise, `reverse` is called again with the address of the subarray beginning at element `sPtr[1]`, and character `sPtr[0]` is output with `putchar` when the recursive call is completed.
- ▶ The order of the two statements in the `else` portion of the `if` statement causes `reverse` to walk to the terminating null character of the string before a character is printed.
- ▶ As the recursive calls are completed, the characters are output in reverse order.



```
1 // Fig. 8.10: fig08_10.c
2 // Using functions fgets and putchar
3 #include <stdio.h>
4 #define SIZE 80
5
6 void reverse( const char * const sPtr ); // prototype
7
8 int main( void )
9 {
10     char sentence[ SIZE ]; // create char array
11
12     puts( "Enter a line of text:" );
13
14     // use fgets to read line of text
15     fgets( sentence, SIZE, stdin );
16
17     puts( "\nThe line printed backward is:" );
18     reverse( sentence );
19 } // end main
20
```

Fig. 8.10 | Using functions fgets and putchar. (Part I of 3.)



```
21 // recursively outputs characters in string in reverse order
22 void reverse( const char * const sPtr )
23 {
24     // if end of the string
25     if ( '\0' == sPtr[ 0 ] ) { // base case
26         return;
27     } // end if
28     else { // if not end of the string
29         reverse( &sPtr[ 1 ] ); // recursion step
30         putchar( sPtr[ 0 ] ); // use putchar to display character
31     } // end else
32 } // end function reverse
```

Enter a line of text:
Characters and Strings

The line printed backward is:
sgnirtS dna sretcarahC

Fig. 8.10 | Using functions fgets and putchar. (Part 2 of 3.)



```
Enter a line of text:  
able was I ere I saw elba
```

```
The line printed backward is:  
able was I ere I saw elba
```

Fig. 8.10 | Using functions `fgets` and `putchar`. (Part 3 of 3.)



8.5.2 Function `getchar`

- ▶ Figure 8.11 uses functions `getchar` and `puts` to read characters from the standard input into character array `sentence` and display the characters as a string.
- ▶ Function `getchar` reads a character from the *standard input* and returns the character as an integer.
- ▶ Function `puts` takes a string as an argument and displays the string followed by a newline character.
- ▶ The program stops inputting characters when either 79 characters have been read or when `getchar` reads the *newline* character entered by the user to end the line.
- ▶ A *null character* is appended to array `sentence` (line 20) so that the array may be treated as a string.
- ▶ Then, line 24 uses `puts` to display the string contained in `sentence`.



```
1 // Fig. 8.11: fig08_11.c
2 // Using function getchar.
3 #include <stdio.h>
4 #define SIZE 80
5
6 int main( void )
7 {
8     int c; // variable to hold character input by user
9     char sentence[ SIZE ]; // create char array
10    int i = 0; // initialize counter i
11
12    // prompt user to enter line of text
13    puts( "Enter a line of text:" );
14
15    // use getchar to read each character
16    while ( i < SIZE - 1 && ( c = getchar() ) != '\n' ) {
17        sentence[ i++ ] = c;
18    } // end while
19
20    sentence[ i ] = '\0'; // terminate string
21
22    // use puts to display sentence
23    puts( "\nThe line entered was:" );
24    puts( sentence );
25 }
```

Fig. 8.11 | Using function getchar. (Part 1 of 2.)



```
Enter a line of text:  
This is a test.
```

```
The line entered was:  
This is a test.
```

Fig. 8.11 | Using function `getchar`. (Part 2 of 2.)



8.5.3 Function `sprintf`

- ▶ Figure 8.12 uses function `sprintf` to print formatted data into array `S`—an array of characters.
- ▶ The function uses the same conversion specifiers as `printf` (see Chapter 9 for a detailed discussion of formatting).
- ▶ The program inputs an `int` value and a `double` value to be formatted and printed to array `S`.
- ▶ Array `S` is the first argument of `sprintf`.
 - [Note: If your system supports `snprintf_s`, then use that in preference to `sprintf`. If your system doesn't support `snprintf_s` but does support `snprintf`, then use that in preference to `sprintf`. We discuss each of these functions in Appendix F.]



```
1 // Fig. 8.12: fig08_12.c
2 // Using function sprintf
3 #include <stdio.h>
4 #define SIZE 80
5
6 int main( void )
7 {
8     char s[ SIZE ]; // create char array
9     int x; // x value to be input
10    double y; // y value to be input
11
12    puts( "Enter an integer and a double:" );
13    scanf( "%d%lf", &x, &y );
14
15    sprintf( s, "integer:%6d\ndouble:%8.2f", x, y );
16
17    printf( "%s\n%s\n",
18           "The formatted output stored in array s is:", s );
19 }
```

Fig. 8.12 | Using function sprintf. (Part I of 2.)



```
Enter an integer and a double:  
298 87.375  
The formatted output stored in array s is:  
integer:   298  
double:   87.38
```

Fig. 8.12 | Using function `sprintf`. (Part 2 of 2.)



8.5.4 Function `sscanf`

- ▶ Figure 8.13 uses function `sscanf` to read formatted data from character array `S`.
- ▶ The function uses the same conversion specifiers as `scanf`.
- ▶ The program reads an `int` and a `double` from array `s` and stores the values in `x` and `y`, respectively.
- ▶ The values of `x` and `y` are printed.
- ▶ Array `S` is the first argument of `sscanf`.



```
1 // Fig. 8.13: fig08_13.c
2 // Using function sscanf
3 #include <stdio.h>
4
5 int main( void )
6 {
7     char s[] = "31298 87.375"; // initialize array s
8     int x; // x value to be input
9     double y; // y value to be input
10
11     sscanf( s, "%d%lf", &x, &y );
12     printf( "%s\n%s%6d\n%s%8.3f\n",
13            "The values stored in character array s are:",
14            "integer:", x, "double:", y );
15 }
```

The values stored in character array s are:
integer: 31298
double: 87.375

Fig. 8.13 | Using function sscanf.



8.6 String-Manipulation Functions of the String-Handling Library

- ▶ The string-handling library (`<string.h>`) provides many useful functions for manipulating string data (copying strings and concatenating strings), comparing strings, searching strings for characters and other strings, tokenizing strings (separating strings into logical pieces) and determining the length of strings.
- ▶ This section presents the string-manipulation functions of the string-handling library.
- ▶ The functions are summarized in Fig. 8.17.
- ▶ Every function—except for `strncpy`—appends the *null* character to its result.



Function prototype	Function description
--------------------	----------------------

<code>char *strcpy(char *s1, const char *s2)</code>	
---	--

<i>Copies string s2 into array s1. The value of s1 is returned.</i>

<code>char *strncpy(char *s1, const char *s2, size_t n)</code>	
--	--

<i>Copies at most n characters of string s2 into array s1. The value of s1 is returned.</i>

<code>char *strcat(char *s1, const char *s2)</code>	
---	--

<i>Appends string s2 to array s1. The first character of s2 overwrites the terminating null character of s1. The value of s1 is returned.</i>

<code>char *strncat(char *s1, const char *s2, size_t n)</code>	
--	--

<i>Appends at most n characters of string s2 to array s1. The first character of s2 overwrites the terminating null character of s1. The value of s1 is returned.</i>

Fig. 8.14 | String-manipulation functions of the string-handling library.



8.6 String-Manipulation Functions of the String-Handling Library (Cont.)

- ▶ Functions `strncpy` and `strncat` specify a parameter of type `size_t`, which is a type defined by the C standard as the integral type of the value returned by operator `sizeof`.
- ▶ Function `strcpy` copies its second argument (a string) into its first argument—a character array that *you must ensure is large enough* to store the string and its terminating null character, which is also copied.



8.6 String-Manipulation Functions of the String-Handling Library (Cont.)

- ▶ Function `strncpy` is equivalent to `strcpy`, except that `strncpy` specifies the number of characters to be copied from the string into the array.
- ▶ *Function `strncpy` does not necessarily copy the terminating null character of its second argument.*
- ▶ *This occurs only if the number of characters to be copied is at least one more than the length of the string.*



8.6 String-Manipulation Functions of the String-Handling Library (Cont.)

- ▶ For example, if `"test"` is the second argument, a terminating null character is written only if the third argument to `strncpy` is at least 5 (four characters in `"test"` plus a terminating null character).
- ▶ If the third argument is larger than 5, null characters are appended to the array until the total number of characters specified by the third argument are written.



Error-Prevention Tip 8.2

When using functions from the string-handling library, include the `<string.h>` header.



Common Programming Error 8.5

Not appending a terminating null character to the first argument of a `strncpy` when the third argument is less than or equal to the length of the string in the second argument.



8.6.1 Functions `strcpy` and `strncpy`

- ▶ Figure 8.15 uses `strcpy` to copy the entire string in array `x` into array `y` and uses `strncpy` to copy the first 14 characters of array `x` into array `z`.
- ▶ A *null character* (`'\0'`) is appended to array `z`, because the call to `strncpy` in the program *does not write a terminating null character* (the third argument is less than the string length of the second argument).



```
1 // Fig. 8.15: fig08_15.c
2 // Using functions strcpy and strncpy
3 #include <stdio.h>
4 #include <string.h>
5 #define SIZE1 25
6 #define SIZE2 15
7
8 int main( void )
9 {
10     char x[] = "Happy Birthday to You"; // initialize char array x
11     char y[ SIZE1 ]; // create char array y
12     char z[ SIZE2 ]; // create char array z
13
14     // copy contents of x into y
15     printf( "%s%s\n%s%s\n",
16         "The string in array x is: ", x,
17         "The string in array y is: ", strcpy( y, x ) );
18
19     // copy first 14 characters of x into z. Does not copy null
20     // character
21     strncpy( z, x, SIZE2 - 1 );
22
23     z[ SIZE2 - 1 ] = '\0'; // terminate string in z
24     printf( "The string in array z is: %s\n", z );
25 }
```

Fig. 8.15 | Using functions strcpy and strncpy. (Part I of 2.)



```
The string in array x is: Happy Birthday to You  
The string in array y is: Happy Birthday to You  
The string in array z is: Happy Birthday
```

Fig. 8.15 | Using functions strcpy and strncpy. (Part 2 of 2.)



8.6.2 Functions `strcat` and `strncat`

- ▶ Function `strcat` appends its second argument (a string) to its first argument (a character array containing a string).
- ▶ The first character of the second argument replaces the null (`'\0'`) that terminates the string in the first argument.
- ▶ *You must ensure that the array used to store the first string is large enough to store the first string, the second string and the terminating null character copied from the second string.*
- ▶ Function `strncat` appends a specified number of characters from the second string to the first string.
- ▶ A terminating null character is appended to the result.
- ▶ Figure 8.16 demonstrates functions `strcat` and `strncat`.



```
1 // Fig. 8.16: fig08_16.c
2 // Using functions strcat and strncat
3 #include <stdio.h>
4 #include <string.h>
5
6 int main( void )
7 {
8     char s1[ 20 ] = "Happy "; // initialize char array s1
9     char s2[] = "New Year "; // initialize char array s2
10    char s3[ 40 ] = ""; // initialize char array s3 to empty
11
12    printf( "s1 = %s\ns2 = %s\n", s1, s2 );
13
14    // concatenate s2 to s1
15    printf( "strcat( s1, s2 ) = %s\n", strcat( s1, s2 ) );
16
17    // concatenate first 6 characters of s1 to s3. Place '\0'
18    // after last character
19    printf( "strncat( s3, s1, 6 ) = %s\n", strncat( s3, s1, 6 ) );
20
21    // concatenate s1 to s3
22    printf( "strcat( s3, s1 ) = %s\n", strcat( s3, s1 ) );
23 }
```

Fig. 8.16 | Using functions strcat and strncat. (Part I of 2.)



```
s1 = Happy  
s2 = New Year  
strcat( s1, s2 ) = Happy New Year  
strncat( s3, s1, 6 ) = Happy  
strcat( s3, s1 ) = Happy Happy New Year
```

Fig. 8.16 | Using functions strcat and strncat. (Part 2 of 2.)

8.7 Comparison Functions of the String-Handling Library



- ▶ This section presents the string-handling library's **string-comparison functions**, **strcmp** and **strncmp**.
- ▶ Fig. 8.17 contains their prototypes and a brief description of each function.



Function prototype	Function description
--------------------	----------------------

<code>int strcmp(const char *s1, const char *s2);</code>	
--	--

	<i>Compares the string s1 with the string s2. The function returns 0, less than 0 or greater than 0 if s1 is equal to, less than or greater than s2, respectively.</i>
--	--

<code>int strncmp(const char *s1, const char *s2, size_t n);</code>	
---	--

	<i>Compares up to n characters of the string s1 with the string s2. The function returns 0, less than 0 or greater than 0 if s1 is equal to, less than or greater than s2, respectively.</i>
--	--

Fig. 8.17 | String-comparison functions of the string-handling library.



8.7 Comparison Functions of the String-Handling Library (Cont.)

- ▶ Figure 8.18 compares three strings using `strcmp` and `strncmp`.
- ▶ Function `strcmp` compares its first string argument with its second string argument, character by character.
- ▶ The function returns 0 if the strings are equal, a *negative value* if the first string is less than the second string and a *positive value* if the first string is greater than the second string.
- ▶ Function `strncmp` is equivalent to `strcmp`, except that `strncmp` compares up to a specified number of characters.
- ▶ Function `strncmp` does *not* compare characters following a null character in a string.
- ▶ The program prints the integer value returned by each function call.



```
1 // Fig. 8.18: fig08_18.c
2 // Using functions strcmp and strncmp
3 #include <stdio.h>
4 #include <string.h>
5
6 int main( void )
7 {
8     const char *s1 = "Happy New Year"; // initialize char pointer
9     const char *s2 = "Happy New Year"; // initialize char pointer
10    const char *s3 = "Happy Holidays"; // initialize char pointer
11
12    printf("%s%s\n%s%s\n%s%s\n\n%s%2d\n%s%2d\n%s%2d\n\n",
13        "s1 = ", s1, "s2 = ", s2, "s3 = ", s3,
14        "strcmp(s1, s2) = ", strcmp( s1, s2 ),
15        "strcmp(s1, s3) = ", strcmp( s1, s3 ),
16        "strcmp(s3, s1) = ", strcmp( s3, s1 ) );
17
18    printf("%s%2d\n%s%2d\n%s%2d\n",
19        "strncmp(s1, s3, 6) = ", strncmp( s1, s3, 6 ),
20        "strncmp(s1, s3, 7) = ", strncmp( s1, s3, 7 ),
21        "strncmp(s3, s1, 7) = ", strncmp( s3, s1, 7 ) );
22 }
```

Fig. 8.18 | Using functions strcmp and strncmp. (Part I of 2.)



```
s1 = Happy New Year  
s2 = Happy New Year  
s3 = Happy Holidays  
  
strcmp(s1, s2) = 0  
strcmp(s1, s3) = 1  
strcmp(s3, s1) = -1  
  
strncmp(s1, s3, 6) = 0  
strncmp(s1, s3, 7) = 6  
strncmp(s3, s1, 7) = -6
```

Fig. 8.18 | Using functions `strcmp` and `strncmp`. (Part 2 of 2.)



Common Programming Error 8.6

Assuming that `strcmp` and `strncmp` return 1 when their arguments are equal is a logic error. Both functions return 0 (strangely, the equivalent of C's false value) for equality. Therefore, when comparing two strings for equality, the result of function `strcmp` or `strncmp` should be compared with 0 to determine whether the strings are equal.

8.7 Comparison Functions of the String-Handling Library (Cont.)



- ▶ To understand just what it means for one string to be “greater than” or “less than” another, consider the process of alphabetizing a series of last names.
- ▶ The reader would, no doubt, place “Jones” before “Smith,” because the first letter of “Jones” comes before the first letter of “Smith” in the alphabet.



8.7 Comparison Functions of the String-Handling Library (Cont.)

- ▶ But the alphabet is more than just a list of 26 letters—it's an ordered list of characters.
- ▶ Each letter occurs in a specific position within the list.
- ▶ “Z” is more than merely a letter of the alphabet; “Z” is specifically the 26th letter of the alphabet.
- ▶ How do the string comparison functions know that one particular letter comes before another?
- ▶ All characters are represented inside the computer as **numeric codes** in character sets such as ASCII and Unicode; when the computer compares two strings, it actually compares the numeric codes of the characters in the strings.



8.8 Search Functions of the String-Handling Library

- ▶ This section presents the functions of the string-handling library used to search strings for characters and other strings.
- ▶ The functions are summarized in Fig. 8.19.
- ▶ The functions `strcspn` and `strspn` return `size_t`.
- ▶ [*Note:* Function `strtok` has a more secure version described in optional Annex K of the C11 standard. We mention this in the Secure C Programming section of this chapter and in Appendix F.]



Function prototypes and descriptions

char *strchr(**const char** *s, **int** c);

Locates the first occurrence of character c in string s. If c is found, a pointer to c in s is returned. Otherwise, a NULL pointer is returned.

size_t strcspn(**const char** *s1, **const char** *s2);

Determines and returns the length of the initial segment of string s1 consisting of characters *not* contained in string s2.

size_t strspn(**const char** *s1, **const char** *s2);

Determines and returns the length of the initial segment of string s1 consisting *only* of characters contained in string s2.

char *strpbrk(**const char** *s1, **const char** *s2);

Locates the first occurrence in string s1 of any character in string s2. If a character from string s2 is found, a pointer to the character in string s1 is returned. Otherwise, a NULL pointer is returned.

Fig. 8.19 | Search functions of the string-handling library. (Part 1 of 2.)



Function prototypes and descriptions

char *strrchr(**const char** *s, **int** c);

Locates the last occurrence of c in string s. If c is found, a pointer to c in string s is returned. Otherwise, a NULL pointer is returned.

char *strstr(**const char** *s1, **const char** *s2);

Locates the first occurrence in string s1 of string s2. If the string is found, a pointer to the string in s1 is returned. Otherwise, a NULL pointer is returned.

char *strtok(**char** *s1, **const char** *s2);

A sequence of calls to strtok breaks string s1 into *tokens*—logical pieces such as words in a line of text—separated by characters contained in string s2. The first call contains s1 as the first argument, and subsequent calls to continue tokenizing the same string contain NULL as the first argument. A pointer to the current token is returned by each call. If there are no more tokens when the function is called, NULL is returned.

Fig. 8.19 | Search functions of the string-handling library. (Part 2 of 2.)



8.8.1 Function `strchr`

- ▶ Function `strchr` searches for the *first occurrence* of a character in a string.
- ▶ If the character is found, `strchr` returns a pointer to the character in the string; otherwise, `strchr` returns `NULL`.
- ▶ Figure 8.20 searches for the first occurrences of 'a' and 'z' in "This is a test".



```
1 // Fig. 8.20: fig08_20.c
2 // Using function strchr
3 #include <stdio.h>
4 #include <string.h>
5
6 int main( void )
7 {
8     const char *string = "This is a test"; // initialize char pointer
9     char character1 = 'a'; // initialize character1
10    char character2 = 'z'; // initialize character2
11
12    // if character1 was found in string
13    if ( strchr( string, character1 ) != NULL ) {
14        printf( "'%c' was found in \"%s\".\n",
15              character1, string );
16    } // end if
17    else { // if character1 was not found
18        printf( "'%c' was not found in \"%s\".\n",
19              character1, string );
20    } // end else
21}
```

Fig. 8.20 | Using function strchr. (Part I of 2.)



```
22 // if character2 was found in string
23 if ( strchr( string, character2 ) != NULL ) {
24     printf( "\"'%c\' was found in \"%s\".\n",
25         character2, string );
26 } // end if
27 else { // if character2 was not found
28     printf( "\"'%c\' was not found in \"%s\".\n",
29         character2, string );
30 } // end else
31 } // end main
```

'a' was found in "This is a test".
'z' was not found in "This is a test".

Fig. 8.20 | Using function strchr. (Part 2 of 2.)



8.8.2 Function `strcspn`

- ▶ Function `strcspn` (Fig. 8.21) determines the length of the initial part of the string in its first argument that does *not* contain any characters from the string in its second argument.
- ▶ The function returns the length of the segment.



```
1 // Fig. 8.21: fig08_21.c
2 // Using function strcspn
3 #include <stdio.h>
4 #include <string.h>
5
6 int main( void )
7 {
8     // initialize two char pointers
9     const char *string1 = "The value is 3.14159";
10    const char *string2 = "1234567890";
11
12    printf( "%s%s\n%s%s\n\n%s\n%s%u\n",
13           "string1 = ", string1, "string2 = ", string2,
14           "The length of the initial segment of string1",
15           "containing no characters from string2 = ",
16           strcspn( string1, string2 ) );
17 }
```

```
string1 = The value is 3.14159
string2 = 1234567890
```

```
The length of the initial segment of string1
containing no characters from string2 = 13
```

Fig. 8.21 | Using function strcspn.



8.8.3 Function `strpbrk`

- ▶ Function `strpbrk` searches its first string argument for the *first occurrence* of any character in its second string argument.
- ▶ If a character from the second argument is found, `strpbrk` returns a pointer to the character in the first argument; otherwise, `strpbrk` returns `NULL`.
- ▶ Figure 8.22 shows a program that locates the first occurrence in `string1` of any character from `string2`.



```
1 // Fig. 8.22: fig08_22.c
2 // Using function strpbrk
3 #include <stdio.h>
4 #include <string.h>
5
6 int main( void )
7 {
8     const char *string1 = "This is a test"; // initialize char pointer
9     const char *string2 = "beware"; // initialize char pointer
10
11     printf( "%s\\'%s\\'\\n'%c'%s\\n\\'%s\\'\\n",
12            "Of the characters in ", string2,
13            *strpbrk( string1, string2 ),
14            " appears earliest in ", string1 );
15 }
```

Of the characters in "beware"
'a' appears earliest in
"This is a test"

Fig. 8.22 | Using function strpbrk.



8.8.4 Function `strrchr`

- ▶ Function `strrchr` searches for the *last occurrence* of the specified character in a string.
- ▶ If the character is found, `strrchr` returns a pointer to the character in the string; otherwise, `strrchr` returns `NULL`.
- ▶ Figure 8.23 shows a program that searches for the last occurrence of the character 'z' in the string "A zoo has many animals including zebras."



```
1 // Fig. 8.23: fig08_23.c
2 // Using function strrchr
3 #include <stdio.h>
4 #include <string.h>
5
6 int main( void )
7 {
8     // initialize char pointer
9     const char *string1 = "A zoo has many animals including zebras";
10
11     int c = 'z'; // character to search for
12
13     printf( "%s\n%s'%c'%s\\\"%s\\\"\\n",
14         "The remainder of string1 beginning with the",
15         "last occurrence of character ", c,
16         " is: ", strrchr( string1, c ) );
17 }
```

The remainder of string1 beginning with the
last occurrence of character 'z' is: "zebras"

Fig. 8.23 | Using function strrchr.



8.8.5 Function `strspn`

- ▶ Function `strspn` (Fig. 8.24) determines the length of the *initial part* of the string in its first argument that contains only characters from the string in its second argument.
- ▶ The function returns the length of the segment.



```
1 // Fig. 8.24: fig08_24.c
2 // Using function strspn
3 #include <stdio.h>
4 #include <string.h>
5
6 int main( void )
7 {
8     // initialize two char pointers
9     const char *string1 = "The value is 3.14159";
10    const char *string2 = "ae hi lsTuv";
11
12    printf( "%s%s\n%s%s\n\n%s\n%s%u\n",
13           "string1 = ", string1, "string2 = ", string2,
14           "The length of the initial segment of string1",
15           "containing only characters from string2 = ",
16           strspn( string1, string2 ) );
17 }
```

```
string1 = The value is 3.14159
string2 = ae hi lsTuv
```

```
The length of the initial segment of string1
containing only characters from string2 = 13
```

Fig. 8.24 | Using function strspn.



8.8.6 Function `strstr`

- ▶ Function `strstr` searches for the *first occurrence* of its second string argument in its first string argument.
- ▶ If the second string is found in the first string, a pointer to the location of the string in the first argument is returned.
- ▶ Figure 8.25 uses `strstr` to find the string "def" in the string "abcdefabcdef".



```
1 // Fig. 8.25: fig08_25.c
2 // Using function strstr
3 #include <stdio.h>
4 #include <string.h>
5
6 int main( void )
7 {
8     const char *string1 = "abcdefabcdef"; // string to search
9     const char *string2 = "def"; // string to search for
10
11     printf( "%s\n%s\n\n%s\n%s\n",
12            "string1 = ", string1, "string2 = ", string2,
13            "The remainder of string1 beginning with the",
14            "first occurrence of string2 is: ",
15            strstr( string1, string2 ) );
16 }
```

```
string1 = abcdefabcdef
string2 = def
```

```
The remainder of string1 beginning with the
first occurrence of string2 is: defabcdef
```

Fig. 8.25 | Using function strstr.



8.8.7 Function strtok

- ▶ Function `strtok` (Fig. 8.26) is used to break a string into a series of `tokens`.
- ▶ A token is a sequence of characters separated by `delimiters` (usually *spaces* or *punctuation marks*, but a delimiter can be *any character*).
- ▶ For example, in a line of text, each word can be considered a token, and the spaces and punctuation separating the words can be considered delimiters.



```
1 // Fig. 8.26: fig08_26.c
2 // Using function strtok
3 #include <stdio.h>
4 #include <string.h>
5
6 int main( void )
7 {
8     // initialize array string
9     char string[] = "This is a sentence with 7 tokens";
10    char *tokenPtr; // create char pointer
11
12    printf( "%s\n%s\n\n%s\n",
13           "The string to be tokenized is:", string,
14           "The tokens are:" );
15
16    tokenPtr = strtok( string, " " ); // begin tokenizing sentence
17
18    // continue tokenizing sentence until tokenPtr becomes NULL
19    while ( tokenPtr != NULL ) {
20        printf( "%s\n", tokenPtr );
21        tokenPtr = strtok( NULL, " " ); // get next token
22    } // end while
23 }
```

Fig. 8.26 | Using function strtok. (Part I of 2.)



```
The string to be tokenized is:  
This is a sentence with 7 tokens
```

```
The tokens are:
```

```
This  
is  
a  
sentence  
with  
7  
tokens
```

Fig. 8.26 | Using function strtok. (Part 2 of 2.)



8.8.7 Function strtok (Cont.)

- ▶ Multiple calls to `strtok` are required to tokenize a string—i.e., break it into tokens (assuming that the string contains more than one token).
- ▶ The first call to `strtok` contains two arguments: a string to be tokenized, and a string containing characters that separate the tokens.
- ▶ In line 16, the statement

```
    // begin tokenizing sentence  
    tokenPtr = strtok( string, " " );
```

assigns `tokenPtr` a pointer to the first token in `string`.



8.8.7 Function strtok (Cont.)

- ▶ The second argument, " ", indicates that tokens are separated by spaces.
- ▶ Function `strtok` searches for the first character in `string` that's not a delimiting character (space).
- ▶ This begins the first token.
- ▶ The function then finds the next delimiting character in the string and *replaces it with a null (' \0 ') character* to terminate the current token.
- ▶ Function `strtok` saves a pointer to the next character following the token in `string` and returns a pointer to the current token.



8.8.7 Function strtok (Cont.)

- ▶ Subsequent `strtok` calls in line 21 continue tokenizing `string`.
- ▶ These calls contain *NULL as their first argument*.
- ▶ The `NULL` argument indicates that the call to `strtok` should continue tokenizing from the location in `string` saved by the last call to `strtok`.
- ▶ If no tokens remain when `strtok` is called, `strtok` returns `NULL`.



8.8.7 Function strtok (Cont.)

- ▶ You can change the delimiter string in each new call to `strtok`.
- ▶ Figure 8.26 uses `strtok` to tokenize the string "This is a sentence with 7 tokens".
- ▶ Each token is printed separately.
- ▶ Function `strtok` *modifies the input string* by placing '`\0`' at the end of each token; therefore, a *copy* of the string should be made if the string will be used again in the program after the calls to `strtok`.



8.9 Memory Functions of the String-Handling Library

- ▶ The string-handling library functions presented in this section manipulate, compare and search blocks of memory.
- ▶ The functions treat blocks of memory as character arrays and can manipulate any block of data.
- ▶ Figure 8.27 summarizes the memory functions of the string-handling library.
- ▶ In the function discussions, “object” refers to a block of data.



Function prototype	Function description
--------------------	----------------------

void *memcpy(void *s1, const void *s2, size_t n);	
--	--

<i>Copies n characters from the object pointed to by s2 into the object pointed to by s1. A pointer to the resulting object is returned.</i>	
---	--

void *memmove(void *s1, const void *s2, size_t n);	
---	--

<i>Copies n characters from the object pointed to by s2 into the object pointed to by s1. The copy is performed as if the characters were first copied from the object pointed to by s2 into a <i>temporary array</i> and then from the temporary array into the object pointed to by s1. A pointer to the resulting object is returned.</i>	
---	--

int memcmp(const void *s1, const void *s2, size_t n);	
--	--

<i>Compares the first n characters of the objects pointed to by s1 and s2. The function returns 0, less than 0 or greater than 0 if s1 is equal to, less than or greater than s2.</i>	
--	--

Fig. 8.27 | Memory functions of the string-handling library. (Part 1 of 2.)



Function prototype

Function description

void *memchr(**const void** *s, **int** c, size_t n);

Locates the first occurrence of c (converted to unsigned char) in the first n characters of the object pointed to by s. If c is found, a pointer to c in the object is returned. Otherwise, NULL is returned.

void *memset(**void** *s, **int** c, size_t n);

Copies c (converted to unsigned char) into the first n characters of the object pointed to by s. A pointer to the result is returned.

Fig. 8.27 | Memory functions of the string-handling library. (Part 2 of 2.)



8.9 Memory Functions of the String-Handling Library (Cont.)

- ▶ The pointer parameters are declared `void *` so they can manipulate memory for any data type.
- ▶ In Chapter 7, we saw that a pointer to any data type can be assigned directly to a pointer of type `void *`, and a pointer of type `void *` can be assigned directly to a pointer to any data type.
- ▶ So, these functions can receive pointers to any data type.
- ▶ Because a `void *` pointer cannot be dereferenced, each function receives a size argument that specifies the number of characters (bytes) the function will process.
- ▶ For simplicity, the examples in this section manipulate character arrays (blocks of characters).



8.9.1 Function `memcpy`

- ▶ Function `memcpy` copies a specified number of characters from the object pointed to by its second argument into the object pointed to by its first argument.
- ▶ The function can receive a pointer to any type of object.
- ▶ The result of this function is *undefined* if the two objects overlap in memory (i.e., if they are parts of the same object)—in such cases, use `memmove`.
- ▶ Figure 8.31 uses `memcpy` to copy the string in array `s2` to array `s1`.



Performance Tip 8.1

`memcpy` is more efficient than `strcpy` when you know the size of the string you are copying.



```
1 // Fig. 8.28: fig08_28.c
2 // Using function memcpy
3 #include <stdio.h>
4 #include <string.h>
5
6 int main( void )
7 {
8     char s1[ 17 ]; // create char array s1
9     char s2[] = "Copy this string"; // initialize char array s2
10
11     memcpy( s1, s2, 17 );
12     printf( "%s\n%s\n"%s"\n",
13         "After s2 is copied into s1 with memcpy,",
14         "s1 contains ", s1 );
15 }
```

After s2 is copied into s1 with memcpy,
s1 contains "Copy this string"

Fig. 8.28 | Using function memcpy.



8.9.2 Function memmove

- ▶ Function **memmove**, like **memcpy**, *copies a specified number of bytes* from the object pointed to by its second argument into the object pointed to by its first argument.
- ▶ Copying is performed as if the bytes were copied from the second argument into a temporary character array, then copied from the temporary array into the first argument.
- ▶ This allows characters from one part of a string to be copied into another part of the *same* string.
- ▶ Figure 8.29 uses **memmove** to copy the last 10 bytes of array **X** into the first 10 bytes of array **X**.



Common Programming Error 8.7

String-manipulation functions other than `memmove` that copy characters have undefined results when copying takes place between parts of the same string.



```
1 // Fig. 8.29: fig08_29.c
2 // Using function memmove
3 #include <stdio.h>
4 #include <string.h>
5
6 int main( void )
7 {
8     char x[] = "Home Sweet Home"; // initialize char array x
9
10    printf( "%s\n", "The string in array x before memmove is: ", x );
11    printf( "%s\n", "The string in array x after memmove is: ",
12           (char *) memmove( x, &x[ 5 ], 10 ) );
13 }
```

The string in array x before memmove is: Home Sweet Home
The string in array x after memmove is: Sweet Home Home

Fig. 8.29 | Using function memmove.



8.9.3 Function memcmp

- ▶ Function `memcmp` (Fig. 8.30) *compares the specified number of characters* of its first argument with the corresponding characters of its second argument.
- ▶ The function returns a value greater than 0 if the first argument is *greater than* the second, returns 0 if the arguments are equal and returns a value less than 0 if the first argument is less than the second.



```
1 // Fig. 8.30: fig08_30.c
2 // Using function memcmp
3 #include <stdio.h>
4 #include <string.h>
5
6 int main( void )
7 {
8     char s1[] = "ABCDEFGG"; // initialize char array s1
9     char s2[] = "ABCDXYZ"; // initialize char array s2
10
11     printf( "%s%s\n%s%s\n\n%s%2d\n%s%2d\n%s%2d\n",
12            "s1 = ", s1, "s2 = ", s2,
13            "memcmp( s1, s2, 4 ) = ", memcmp( s1, s2, 4 ),
14            "memcmp( s1, s2, 7 ) = ", memcmp( s1, s2, 7 ),
15            "memcmp( s2, s1, 7 ) = ", memcmp( s2, s1, 7 ) );
16 }
```

```
s1 = ABCDEFG
s2 = ABCDXYZ
```

```
memcmp( s1, s2, 4 ) = 0
memcmp( s1, s2, 7 ) = -1
memcmp( s2, s1, 7 ) = 1
```

Fig. 8.30 | Using function memcmp.



8.9.4 Function `memchr`

- ▶ Function `memchr` searches for the *first occurrence of a byte*, represented as `unsigned char`, in the specified number of bytes of an object.
- ▶ If the byte is found, a pointer to the byte in the object is returned; otherwise, a `NULL` pointer is returned.
- ▶ Figure 8.31 searches for the character (byte) `'r'` in the string `"This is a string"`.



```
1 // Fig. 8.31: fig08_31.c
2 // Using function memchr
3 #include <stdio.h>
4 #include <string.h>
5
6 int main( void )
7 {
8     const char *s = "This is a string"; // initialize char pointer
9
10    printf( "%s\\'%c\\'%s\\'%s\\'\\n",
11           "The remainder of s after character ", 'r',
12           " is found is ", (char *) memchr( s, 'r', 16 ) );
13 }
```

The remainder of s after character 'r' is found is "ring"

Fig. 8.31 | Using function memchr.



8.9.5 Function `memset`

- ▶ Function `memset` copies the value of the byte in its second argument into the first n bytes of the object pointed to by its first argument, where n is specified by the third argument.
- ▶ Figure 8.32 uses `memset` to copy 'b' into the first 7 bytes of `string1`.



Performance Tip 8.2

Use `memset` to set an array's values to 0 rather than looping through the array's elements and assigning 0 to each element. For example, in Fig. 6.3, we could have initialized the 10-element array `n` with `memset(n, 0, 10) ;`. Many hardware architectures have a block copy or clear instruction that the compiler can use to optimize `memset` for high performance zeroing of memory.



```
1 // Fig. 8.32: fig08_32.c
2 // Using function memset
3 #include <stdio.h>
4 #include <string.h>
5
6 int main( void )
7 {
8     char string1[ 15 ] = "BBBBBBBBBBBBBBB"; // initialize string1
9
10    printf( "string1 = %s\n", string1 );
11    printf( "string1 after memset = %s\n",
12           (char *) memset( string1, 'b', 7 ) );
13 }
```

```
string1 = BBBBBBBBBBBBBB
string1 after memset = bbbbbbbBBBBBBB
```

Fig. 8.32 | Using function memset.



8.10 Other Functions of the String-Handling Library

- ▶ The two remaining functions of the string-handling library are `strerror` and `strlen`.
- ▶ Figure 8.33 summarizes the `strerror` and `strlen` functions.



Function prototype	Function description
<code>char *strerror(int errornum);</code>	Maps <code>errornum</code> into a full text string in a compiler- and locale-specific manner (e.g. the message may appear in different languages based on its location). A pointer to the string is returned.
<code>size_t strlen(const char *s);</code>	Determines the length of string <code>s</code> . The number of characters preceding the terminating null character is returned.

Fig. 8.33 | Other functions of the string-handling library.



8.10.1 Function `strerror`

- ▶ Function `strerror` takes an error number and creates an error message string.
- ▶ A pointer to the string is returned.
- ▶ Figure 8.34 demonstrates `strerror`.



```
1 // Fig. 8.34: fig08_34.c
2 // Using function strerror
3 #include <stdio.h>
4 #include <string.h>
5
6 int main( void )
7 {
8     printf( "%s\n", strerror( 2 ) );
9 } // end main
```

No such file or directory

Fig. 8.34 | Using function strerror.



8.10.2 Function `strlen`

- ▶ Function `strlen` takes a string as an argument and returns the number of characters in the string—the terminating null character is not included in the length.
- ▶ Figure 8.35 demonstrates function `strlen`.



```
1 // Fig. 8.35: fig08_35.c
2 // Using function strlen
3 #include <stdio.h>
4 #include <string.h>
5
6 int main( void )
7 {
8     // initialize 3 char pointers
9     const char *string1 = "abcdefghijklmnopqrstuvwxyz";
10    const char *string2 = "four";
11    const char *string3 = "Boston";
12
13    printf("%s\\\"%s\\\"%s%u\\n%s\\\"%s\\\"%s%u\\n%s\\\"%s\\\"%s%u\\n",
14        "The length of ", string1, " is ", strlen( string1 ),
15        "The length of ", string2, " is ", strlen( string2 ),
16        "The length of ", string3, " is ", strlen( string3 ) );
17 }
```

The length of "abcdefghijklmnopqrstuvwxyz" is 26
The length of "four" is 4
The length of "Boston" is 6

Fig. 8.35 | Using function strlen.



8.11 Secure C Programming

Secure String-Processing Functions

- ▶ In this chapter, we presented functions `sprintf`, `strcpy`, `strncpy`, `strcat`, `strncat`, `strtok`, `strlen`, `memcpy`, `memmove` and `memset`.
- ▶ More secure versions of these and many other string-processing and input/output functions are described by the C11 standard's optional Annex K.
- ▶ If your C compiler supports Annex K, you should use the secure versions of these functions.



8.11 Secure C Programming (Cont.)

- ▶ Among other things, the more secure versions help prevent buffer overflows by requiring an additional parameter that specifies the number of elements in the target array and by ensuring that pointer arguments are non-NULL.



8.11 Secure C Programming (Cont.)

Reading Numeric Inputs and Input Validation

- ▶ It's important to validate the data that you input into a program.
- ▶ For example, when you ask the user to enter an integer in the range 1–100 then attempt to read that integer using `scanf`, there are several possible problems.
- ▶ The user could enter an integer that's outside the program's required range, an integer that's outside the allowed range for integers on that computer, a non-integer numeric value or a non-numeric value.



8.11 Secure C Programming (Cont.)

- ▶ You can use various functions that you learned in this chapter to fully validate such input. For example, you could
 - use `fgets` to read the input as a line of text
 - convert the string to a number using `strtol` and ensure that the conversion was successful, then
 - ensure that the value is in range.
- ▶ For more information and techniques for converting input to numeric values, see CERT guideline INT05-C at www.securecoding.cert.org.